

- ✓ Lesson objectives
- ✓ Key concepts (highlighted keywords and clear definitions)
- ✓ Detailed explanations
- ✓ Hands-on practicals (step-by-step)
- ✓ Classwork and reflective exercises
- ✓ Summary for assessment

Module 1: Understanding the JavaScript Concurrency Model

Learning Objectives

By the end of this module, learners should be able to:

1. Define and explain **Concurrency** and **Parallelism**.
2. Describe how **JavaScript handles concurrency** even though it's **single-threaded**.
3. Identify and explain the components of the **JavaScript Concurrency Model**, including:
 - **Call Stack**
 - **Web APIs**
 - **Callback Queue / Task Queue**
 - **Event Loop**
4. Demonstrate how JavaScript executes synchronous and asynchronous tasks using simple examples.

1. Introduction

When you run a JavaScript program, it seems like the code can do many things at once — like responding to a button click while fetching data from a server.

However, **JavaScript is a single-threaded language**, which means it can only execute **one task at a time** on a single main thread.

So how does JavaScript handle multiple activities without freezing the browser? 🤔

It uses a **Concurrency Model** based on **asynchronous programming** and an **Event Loop**

mechanism.



2. Key Terms and Definitions

Keyword	Explanation	Simple Definition
Single-threaded	JavaScript runs code in one sequence — one statement at a time — in a single main thread.	Runs one task at a time.
Concurrency	The ability of JavaScript to manage multiple tasks so that they <i>appear</i> to run simultaneously without blocking each other.	Managing multiple tasks efficiently at once.
Parallelism	True simultaneous execution of multiple tasks on different CPU cores or threads.	Running multiple tasks at the same time.
Call Stack	A structure that keeps track of function calls. When a function is invoked, it's added to the stack; when it finishes, it's removed.	A stack that stores and executes active functions.
Web APIs	Browser-provided features (like <code>setTimeout</code> , <code>fetch</code> , DOM events) that handle asynchronous operations outside the main thread.	External features that handle async tasks.
Callback Queue (Task Queue)	A waiting line that stores completed async tasks, ready to be pushed back to the call stack for execution.	A queue for tasks waiting to be executed.
Event Loop	The mechanism that continuously checks if the call stack is empty and moves tasks from the queue into the stack.	A controller that manages when async code runs.



3. How JavaScript Executes Code

Let's break down how JavaScript handles tasks step by step:

◆ Step 1: The Call Stack

When you call a function, it's pushed onto the **Call Stack**.

When that function finishes, it's popped off the stack.

Example:

```
function sayHello() {  
  console.log("Hello");  
}  
sayHello();
```

Execution Process:

1. `sayHello()` is added to the Call Stack.
2. `console.log("Hello")` runs.
3. The function is removed from the Call Stack.

🔴 Key Point:

Only **one function runs at a time** in the Call Stack.

◆ Step 2: Web APIs

When you call asynchronous functions like `setTimeout`, they are **handed over to the Web API environment** (not run immediately).

Example:

```
setTimeout(() => {  
  console.log("Task done!");  
}, 2000);
```

What happens internally:

1. `setTimeout()` goes to the **Web API**.
2. The timer runs outside the main thread.
3. After 2 seconds, the callback (`console.log("Task done!")`) moves to the **Task Queue**.

◆ Step 3: Callback Queue

After Web APIs finish their task, they send the callback functions into the **Callback Queue** — waiting to be executed.

However, they don't execute immediately.

They wait for the **Call Stack** to be empty.

◆ Step 4: Event Loop

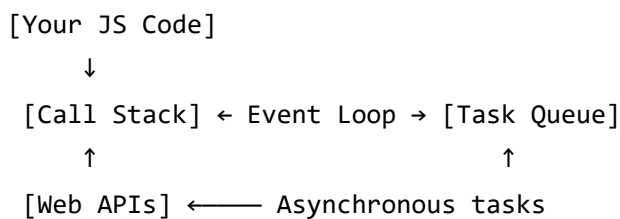
The **Event Loop** constantly monitors:

- The **Call Stack**
- The **Task Queue**

If the stack is empty, the Event Loop picks the first function from the queue and pushes it into the stack for execution.

✚ How They Work Together

Let's visualize:



💡 Explanation:

1. Synchronous tasks run directly in the **Call Stack**.
2. Asynchronous tasks go to **Web APIs**.
3. Completed async tasks move to the **Task Queue**.
4. **Event Loop** checks if the stack is free and moves tasks back for execution.



4. Hands-on Practical



Practical 1.1: Understanding Synchronous Execution

```
console.log("Start");
console.log("Middle");
console.log("End");
```

Expected Output:

Start
Middle
End

Explanation:

This code runs line-by-line in the **Call Stack** with no delay — completely synchronous.

Practical 1.2: Asynchronous Execution with **setTimeout**

```
console.log("Start");

setTimeout(() => {
  console.log("Inside Timeout");
}, 2000);

console.log("End");
```

Expected Output:

Start
End
Inside Timeout

Explanation:

- `setTimeout` is handled by the **Web API**, which runs independently.
- After 2 seconds, the callback moves to the **Task Queue**.
- The **Event Loop** waits until the **Call Stack** is empty before executing it.

Practical 1.3: Multiple Async Tasks

```
console.log("A");

setTimeout(() => console.log("B"), 1000);
setTimeout(() => console.log("C"), 0);

console.log("D");
```

Expected Output:

A
D
C
B

 Explanation:

- A and D run immediately.
- Both `setTimeout` functions are sent to Web APIs.
- The `0 ms` timer (C) is queued first but still waits for the stack to clear.
- B (1 second) executes last.



5. Classwork and Exercises



Classwork 1: Understanding Order of Execution

Predict the output of the following:

```
console.log("1");  
setTimeout(() => console.log("2"), 0);  
Promise.resolve().then(() => console.log("3"));  
console.log("4");
```

- 1 Write down your prediction.
- 2 Run the code in your browser console.
- 3 Compare results and explain why.

Expected Output:

1
4
3
2

 Explanation:

- Promises are **microtasks** and run **before** macrotasks (like `setTimeout`).



Classwork 2: Drawing Task Flow

Draw a simple diagram showing:

- Call Stack
- Web API
- Task Queue
- Event Loop

Label how `setTimeout` or `fetch()` move between these sections.



Classwork 3: Explanation Challenge

Answer these:

1. What does it mean that JavaScript is “single-threaded”?
2. What is the role of the **Event Loop**?
3. Why doesn't `setTimeout(..., 0)` execute immediately?



6. Summary

Concept	Explanation
Concurrency	Running multiple tasks efficiently without blocking the main thread.
Event Loop	Keeps checking if the call stack is empty and moves queued tasks for execution.
Call Stack	Runs one function at a time.
Web APIs	Handle asynchronous tasks like timers, network calls, or DOM events.
Task Queue	Stores completed async callbacks, waiting to run.



Core Understanding:

JavaScript may be **single-threaded**, but its **event-driven model** allows it to behave **asynchronously**.

This combination of the **event loop**, **web APIs**, and **queues** gives JavaScript its **non-blocking power**.



7. Homework / Practice

1. Write code that uses `setTimeout` three times with different delays (e.g., 0ms, 1000ms, 2000ms) and explain the order of output.
2. Research the difference between **microtasks** and **macrotasks**, and give one example of each.
3. Create a short written summary titled “**How the Event Loop Works in My Own Words.**”



End of Module 1